

LUNAR SHOP

Relatorio Tecnico Detalhado

Analise Completa do Sistema E-Commerce

Loja Online Lunar Produtos - Tecnologia, Fragrancias e Variedades

Projeto	Lunar Shop (Lunar Produtos)
Stack Principal	TanStack Start + React 19 + Supabase
Banco de Dados	Supabase (PostgreSQL)
Deploy	Cloudflare Workers
Idioma	Portugues Brasileiro
Data	Setembro 2026

Indice

1. Visao Geral do Projeto
2. Stack Tecnologica
3. Estrutura de Diretorios
4. Rotas e Paginas
5. Componentes
6. Bibliotecas (lib/)
7. Banco de Dados (Supabase)
8. Sistema de Autenticacao
9. Sistema de Carrinho
10. Fluxo de Checkout (WhatsApp)
11. Sistema de Slides (Admin)
12. Estilizacao e Tema
13. Configuracao de Build e Deploy
14. Seguranca e Protecoes
15. Resumo de Funcionalidades

1. Visao Geral do Projeto

O **Lunar Shop** (Lunar Produtos) e uma loja virtual brasileira que comercializa produtos de tecnologia, fragancias, cosmeticos, artesanato e acessorios. O site funciona como um marketplace simples onde usuarios cadastrados podem adicionar produtos, e o checkout e realizado via WhatsApp diretamente com o proprietario da loja.

A aplicacao e um **Single Page Application (SPA)** com rendering no servidor (SSR) via TanStack Start, utilizando banco de dados Supabase (PostgreSQL) e autenticao por e-mail/senha. O deploy e feito em Cloudflare Workers.

Principais Caracteristicas:

- * Catalogo de produtos com imagens, precos, parcelamentos e avaliacoes
- * Sistema de carrinho de compras 100% client-side (localStorage)
- * Checkout via WhatsApp com mensagem pre-preenchida
- * Autenticacao de usuarios com cadastro, login e redefinicao de senha
- * Painel administrativo para gerenciar slides da pagina inicial
- * Usuarios autenticados podem cadastrar e editar produtos
- * Controle de acesso: publico, autenticado, proprietario, administrador
- * Design responsivo com suporte a dark mode
- * Componentes UI baseados em shadcn/ui (40+ componentes)

2. Stack Tecnologica

Camada	Tecnologia	Descricao
Framework	TanStack Start (v3 beta)	Framework full-stack React com SSR e file-based routing
UI Library	React 19.2	Biblioteca principal de interface
Routing	TanStack React Router	Roteamento baseado em arquivos
Estilo	Tailwind CSS v4	CSS utility-first com temas personalizados
Componentes UI	shadcn/ui (New York)	40+ componentes Radix UI
Formularios	react-hook-form + Zod	Formularios com validacao schema-based
Banco de Dados	Supabase (PostgreSQL)	Backend-as-a-Service com auth + DB
Autenticacao	Supabase Auth	Email/senha com recuperacao de senha
Build Tool	Vite 8.1.5	Bundler moderno e rapido
Linguagem	TypeScript 5.8	Tipagem estatica
Icones	Lucide React	Biblioteca de icones SVG
Deploy	Cloudflare Workers	Edge computing global

Camada	Tecnologia	Descricao
SSR Server	Nitro v3 beta	Motor de servidor para TanStack Start
Notifications	Sonner	Sistema de notificacoes toast

3. Estrutura de Diretórios

lunar-shop-main/ src/ start.ts - Middleware TanStack Start (CSRF, erros) server.ts - Entry point do servidor (Cloudflare Workers) router.tsx - Criacao do roteador com QueryClient styles.css - Configuracao Tailwind v4 + tema routeTree.gen.ts - Arvore de rotas auto-gerada routes/ __root.tsx - Layout raiz (providers, 404, erros) index.tsx - Pagina inicial login.tsx - Login cadastro.tsx - Cadastro redefinir-senha.tsx - Redefinicao de senha carrinho.tsx - Carrinho de compras produto.\$id.tsx - Detalhe do produto cadastrar-produto.tsx - Cadastrar produto editar-produto.\$id.tsx - Editar produto admin.slides.tsx - Gerenciar slides (admin) lib/ supabase.ts - Cliente Supabase auth.tsx - Context + Provider de autenticacao cart.ts - Sistema de carrinho (localStorage) utils.ts - Utilitario cn() para classes error-page.ts - Pagina de erro estatica (HTML) error-capture.ts - Captura de erros no servidor data/ products.ts - Tipo Product, categorias, formatador BRL hooks/ use-mobile.tsx - Hook de deteccao mobile components/ store/ Header.tsx - Cabecalho de navegacao Footer.tsx - Rodape ProductCard.tsx - Card de produto Slideshow.tsx - Slideshow da home WhatsAppButton.tsx - Botao WhatsApp ui/ - 40+ componentes shadcn/ui public/ favicon.png robots.txt supabase-schema.sql - Schema do banco de dados

4. Rotas e Paginas

O sistema de roteamento e baseado em arquivos (**TanStack Router**). Cada arquivo em `src/routes/` gera uma rota automaticamente. Parametros dinamicos sao indicados por `$parametro`.

Rota	Arquivo	Descricao
/	index.tsx	Hero slideshow animado, badges de recursos, secao 'Ofertas do dia', listagem de produtos agrupados por categoria
/login	login.tsx	Formulario de email/senha. Modal inline 'Esqueci minha senha' que envia email de redefinicao via Supabase
/cadastro	cadastro.tsx	Formulario de registro com email, senha e confirmacao de senha. Validacao com Zod
/redefinir-senha	redefinir-senha.tsx	Acessado via link magico do Supabase. Extrai token do hash da URL e permite definir nova senha
/carrinho	carrinho.tsx	Lista itens do carrinho com checkboxes, controles de quantidade, resumo do pedido e checkout via WhatsApp
/produto/\$id	produto.\$id.tsx	Exibe imagem, preco, parcelamentos, botao de compra, avaliacoes (localStorage) e produtos relacionados
/cadastrar-produto	cadastrar-produto.tsx	Formulario para autenticados: nome, categoria, preco, imagem (base64), descricao. Protegido por auth
/editar-produto/\$id	editar-produto.\$id.tsx	Edicao/exclusao de produto. Apenas o criador ou admin podem acessar
/admin/slides	admin.slides.tsx	CRUD de slides para o slideshow da home. Acesso restrito a administradores

Layout Raiz (__root.tsx)

O arquivo `__root.tsx` define o layout raiz da aplicacao:

- * Shell HTML completo com meta tags e fontes (Cormorant Garamond + Barlow)
- * Provider do QueryClient (TanStack Query) para cache de dados
- * AuthProvider para gerenciamento de estado de autenticacao
- * Pagina 404 estilizada para rotas nao encontradas
- * Error Boundary para captura de erros de renderizacao

5. Componentes

5.1 Componentes da Loja (store/)

Header.tsx

Barra de navegacao fixa no topo da pagina. Contem:

- * Logo e nome da marca 'Lunar'
- * Barra de pesquisa (placeholder - funcionalidade nao implementada)
- * Badge de localizacao: 'Enviar para todo o Brasil'
- * Exibicao condicional do estado de autenticacao (email + botao sair ou link de login)
- * Icone do carrinho com badge de contagem de itens (via useCart hook)
- * Barra de navegacao por categorias com links ancora para secoes da home
- * Link 'Cadastrar produto' (visivel apenas para usuarios autenticados)
- * Link 'Gerenciar slides' (visivel apenas para administradores)

Footer.tsx

Rodape escuro com informacoes da marca e canais de contato:

- * Branding Lunar com descricao da loja
- * Informacoes de contato: WhatsApp (5587996233203), telefone, Instagram (@luna.ressence)
- * Badges de confianca: seguranca, pagamento (Pix + 12x), frete rapido
- * Linha de copyright

ProductCard.tsx

Card de exibicao de produto usado em grids na pagina inicial e listagens:

- * Imagem do produto (armazenada como base64 no banco)
- * Badge de desconto com percentual calculado
- * Nome do produto (link para pagina de detalhe)
- * Preco antigo riscado e preco atual
- * Informacoes de parcelamento (ex: '12x de R\$ X,XX')
- * Avaliacao em estrelas
- * Badge de frete gratis (quando aplicavel)
- * Botao verde 'Comprar pelo WhatsApp' com mensagem pre-preenchida
- * Botao 'Editar produto' (visivel apenas para usuarios logados)

Slideshow.tsx

Componente de slideshow animado na pagina inicial:

- * Busca slides ativos da tabela 'slides' no Supabase, ordenados por campo 'ordem'
- * Avanco automatico a cada 5 segundos
- * Setas de navegacao esquerda/direita + indicadores de pontos
- * Cada slide exhibe: subtítulo, título, descricao, botao opcional com link

- * Suporte a imagens de fundo com sobreposicao de texto
- * Retorna null se nao houver slides cadastrados

WhatsAppButton.tsx

Botao reutilizavel para compra via WhatsApp:

- * Props: productName, productPrice, productUrl, className, variant (default/small)
- * Constroi link de deep link do WhatsApp com detalhes do produto na mensagem
- * Numero de WhatsApp: 5587996233203

5.2 Componentes UI (shadcn/ui)

O projeto utiliza 40+ componentes da biblioteca shadcn/ui (estilo New York), baseados em primitivas Radix UI. Estes componentes formam a base visual do sistema:

Componente	Funcao
accordion	Conteudo expansivel em formato acordeon
alert	Mensagens de alerta notificacao
alert-dialog	Dialogo de confirmacao modal
avatar	Exibicao de avatar do usuario
badge	Etiquetas de status/categoria
breadcrumb	Navegacao em cascata
button	Botao com variantes (primary, outline, ghost, etc)
calendar	Seletor de datas
card	Container estilizado para conteudo
carousel	Carrossel de itens (Embla)
checkbox	Caixa de selecao
command	Palette de comandos (cmdk)
dialog	Modal de dialogo
drawer	Gaveta lateral (vaul)
dropdown-menu	Menu dropdown de acoes
form	Integracao react-hook-form + Label
input	Campo de entrada de texto
label	Rotulo para campos de formulario
pagination	Navegacao paginada
select	Seletor dropdown
separator	Linha separadora
sheet	Painel lateral deslizante
skeleton	Placeholder de carregamento
slider	Controle deslizante (range)
sonner	Notificacoes toast
switch	Toggle ligar/desligar
table	Tabela de dados
tabs	Abas de conteudo

Componente	Funcao
textarea	Area de texto multilinha
tooltip	Dica flutuante

6. Bibliotecas (lib/)

supabase.ts

Inicializa e exporta o cliente Supabase. Le as variaveis de ambiente VITE_SUPABASE_URL e VITE_SUPABASE_ANON_KEY para configurar a conexao. O projeto esta conectado ao Supabase com ID: ignltvioiyatyyxzeql.

auth.tsx - Sistema de Autenticacao

Exporta **AuthProvider** (React Context Provider) e **useAuth** (hook). O AuthContextType fornece: user, session, loading, isAdmin, signIn, signUp, resetPassword, signOut.

Funcionalidades do AuthProvider:

- * Inicializacao: verifica sessao existente via getSession() + getUser()
- * Inscreve-se em onAuthStateChange para atualizacoes em tempo real (SIGNED_OUT, TOKEN_REFRESHED, etc.)
- * Sessao invalida: faz signOut automatico e limpa estado
- * checkIsAdmin(): verifica app_metadata.role === 'admin' ou user_metadata.role === 'admin'
- * signIn(email, password): chama supabase.auth.signInWithPassword()
- * signUp(email, password): chama supabase.auth.signUp() para registro
- * resetPassword(email): envia email de redefinicao com redirect para /redefinir-senha
- * signOut(): limpa estado local e chama supabase.auth.signOut()

cart.ts - Sistema de Carrinho

Sistema 100% client-side baseado em localStorage. Exporta: CartItem type, addToCart, setQty, removeFromCart, clearCart, useCart.

Detalhes da implementacao:

- * Armazenamento: JSON no localStorage sob chave 'lunar-cart'
- * Modelo de dados: Array de { id: string, qtd: number } (ID do produto + quantidade)
- * Sincronizacao: evento customizado 'lunar-cart-change' + evento nativo 'storage' (cross-tab)
- * addToCart(id, qty): adiciona item ou incrementa quantidade se ja existe
- * setQty(id, qty): define quantidade exata; remove se qty <= 0
- * removeFromCart(id): remove item por ID
- * clearCart(): esvazia o carrinho
- * useCart(): hook React que retorna itens atuais e se atualiza em tempo real

utils.ts

Exporta a funcao cn() - utilitario padrao do shadcn/ui que combina classes Tailwind usando clsx + twMerge.

error-page.ts

Exporta renderErrorPage() que retorna uma string HTML completa para uma pagina de erro estilizada com botoes 'Tentar novamente' (reload) e 'Ir para pagina inicial'.

error-capture.ts

Exporta `describeError()` e `consumeLastCapturedError()`. Captura erros out-of-band para que o servidor recupere stack traces quando o h3 converte excecoes em respostas 500 genericas. Faz monkey-patch em `console.error` para expandir objetos `Error` com cadeias de causa completas. Escuta eventos globais `'error'` e `'unhandledrejection'`. Armazena ultimo erro capturado com TTL de 5 segundos.

7. Banco de Dados (Supabase)

Tabela: produtos

Coluna	Tipo	Constraints	Descricao
id	UUID	PK, auto-generated	ID do produto
nome	TEXT	NOT NULL	Nome do produto
categoria	TEXT	NOT NULL	Categoria (Tecnologia, Artesanato, etc.)
preco	NUMERIC(10,2)	NOT NULL	Preco atual (BRL)
preco_antigo	NUMERIC(10,2)	nullable	Preco original (para exibicao de desconto)
imagem	TEXT	NOT NULL, default ''	Imagem armazenada como base64 data URL
parcelas	INTEGER	NOT NULL, default 1	Quantidade de parcelas
frete_gratis	BOOLEAN	NOT NULL, default false	Flag de frete gratis
avaliacao	NUMERIC(2,1)	NOT NULL, default 5	Avaliacao em estrelas (0-5)
vendidos	INTEGER	NOT NULL, default 0	Quantidade de unidades vendidas
descricao	TEXT	NOT NULL, default ''	Descricao do produto
criado_por	UUID	FK -> auth.users(id)	ID do usuario criador
criado_em	TIMESTAMPTZ	NOT NULL, default now()	Data/hora de criacao

Políticas RLS (Row Level Security) - produtos:

- * SELECT: Publico (qualquer pessoa pode ler)
- * INSERT: Apenas usuarios autenticados, auth.uid() = criado_por
- * UPDATE: Proprietario (auth.uid() = criado_por) OU admin (app_metadata.role = 'admin')
- * DELETE: Proprietario OU admin
- * Indices: idx_produtos_categoria, idx_produtos_criado_por

Tabela: slides

Coluna	Tipo	Constraints	Descricao
id	UUID	PK, auto-generated	ID do slide
titulo	TEXT	NOT NULL, default ''	Titulo do slide
subtitulo	TEXT	NOT NULL, default ''	Subtitulo
descricao	TEXT	NOT NULL, default ''	Descricao do slide
imagem	TEXT	NOT NULL, default ''	Imagem de fundo (base64)
link	TEXT	NOT NULL, default ''	URL do botao CTA

Coluna	Tipo	Constraints	Descricao
texto_botao	TEXT	NOT NULL, default ''	Texto do botao CTA
cor_fundo	TEXT	NOT NULL, default '#0B1120'	Cor de fundo
cor_texto	TEXT	NOT NULL, default '#ffffff'	Cor do texto
ativo	BOOLEAN	NOT NULL, default true	Flag de visibilidade
ordem	INTEGER	NOT NULL, default 0	Ordem de exibicao
criado_em	TIMESTAMPTZ	NOT NULL, default now()	Data/hora de criacao

Políticas RLS - slides:

- * SELECT: Slides ativos são públicos; administradores veem todos
- * INSERT/UPDATE/DELETE: Apenas administradores (app_metadata.role = 'admin')
- * Índice: idxSlidesOrdem

8. Sistema de Autenticacao

O sistema de autenticacao e construido sobre o **Supabase Auth** com autenticacao por email/senha. O fluxo completo inclui:

Fluxo de Registro:

1. Usuario preenche formulario em /cadastro (email, senha, confirmacao de senha)
2. Validacao client-side com Zod (schema de validacao)
3. Chamada a supabase.auth.signUp()
4. Supabase envia email de confirmacao automatico
5. Apos confirmacao, usuario pode fazer login

Fluxo de Login:

1. Usuario acessa /login e preenche email/senha
2. Chamada a supabase.auth.signInWithPassword()
3. Em caso de sucesso: sessao e armazenada, AuthProvider atualiza estado
4. Em caso de erro: mensagem de erro exibida via sonner (toast)

Fluxo de Redefinicao de Senha:

1. Usuario clica 'Esqueci minha senha' na pagina de login
2. Modal inline solicita email do usuario
3. Chamada a supabase.auth.resetPasswordForEmail() com redirect para /redefinir-senha
4. Supabase envia email com link magico
5. Usuario clica no link e e redirecionado para /redefinir-senha
6. Pagina extrai access_token e refresh_token do hash da URL
7. Chamada a supabase.auth.setSession() para autenticar
8. Chamada a supabase.auth.updateUser({password: novaSenha})
9. Sucesso: redireciona para pagina inicial com toast de confirmacao

Niveis de Acesso:

Nivel	Descricao	Exemplos de Rotas
Publico	Qualquer visitante	/ (home), /login, /cadastro, /produto/\$id
Autenticado	Usuario logado	/cadastrar-produto, /carrinho
Proprietario	Criador do produto	/editar-produto/\$id (do proprio produto)
Administrador	Role 'admin' no Supabase	/admin/slides, /editar-produto (qualquer)

9. Sistema de Carrinho

O carrinho de compras é inteiramente **client-side**, armazenado no `localStorage`. Este é um dos sistemas centrais da aplicação.

Arquitetura:

- * Armazenamento: `localStorage` sob chave `'lunar-cart'`
- * Formato: Array JSON de objetos `{ id: string, qtd: number }`
- * Sincronização: Evento customizado `'lunar-cart-change'` para componentes React + evento `'storage'` para sincronização entre abas
- * Hook `useCart()`: retorna lista atualizada de itens, com listeners para atualização em tempo real

API Completa:

Função	Parâmetros	Descrição
<code>addToCart</code>	<code>id: string, qtd?: number (default 1)</code>	Adiciona item ou incrementa quantidade se já existe
<code>setQty</code>	<code>id: string, qtd: number</code>	Define quantidade exata; remove se <code>qtd <= 0</code>
<code>removeFromCart</code>	<code>id: string</code>	Remove item do carrinho por ID
<code>clearCart</code>	<code>()</code>	Esvazia completamente o carrinho
<code>useCart</code>	<code>()</code>	Hook React: retorna itens + se atualiza em tempo real

Página do Carrinho (/carrinho):

- * Carrega detalhes completos dos produtos do Supabase para cada ID no carrinho
- * Suporte a seleção/deseleção de itens individuais com checkboxes
- * Checkbox 'Selecionar todos' para selecionar/deselecionar todos os itens
- * Exibe imagem, nome, preço, controles de quantidade (+/-), botão remover
- * Sidebar de resumo do pedido com total, frete grátis, info de parcelamento (12x)
- * Checkout via WhatsApp: monta mensagem com itens selecionados e abre WhatsApp

Badge no Header:

O componente Header exibe um badge numérico no ícone do carrinho com a contagem total de itens (soma de todas as quantidades).

10. Fluxo de Checkout (WhatsApp)

O checkout não utiliza gateway de pagamento. Todo o processo de compra é feito via WhatsApp, onde o cliente envia o pedido diretamente para o proprietário da loja.

Fluxo Completo:

1. Usuario seleciona itens no carrinho (ou usa todos)
2. Clica em 'Finalizar via WhatsApp'
3. Sistema monta mensagem com: nome do produto, quantidade, preco unitario, subtotal
4. Abre WhatsApp via deep link com mensagem pre-preenchida
5. Numero de destino: 5587996233203
6. Apos envio, itens comprados sao removidos do carrinho automaticamente
7. Toast de confirmacao exibido ao usuario

Formato da Mensagem WhatsApp:

Exemplo de mensagem gerada: ---produto--- Produto: Fone Bluetooth XPTO Qtd: 2 x R\$ 49,90 = R\$ 99,80 --- Total do pedido: R\$ 99,80 Via: Lunar Shop

11. Sistema de Slides (Admin)

O painel administrativo de slides permite gerenciar o conteudo do slideshow exibido na pagina inicial. Acesso restrito a usuarios com role 'admin'.

Funcionalidades:

- * Listar: exhibe todos os slides (ativos e inativos) com titulo, ordem e status
- * Criar: formulario com titulo, subtitulo, descricao, imagem (base64), link, texto do botao, cores de fundo/texto, ativo/inativo, ordem
- * Editar: modal com pre-preenchimento dos dados existentes
- * Excluir: confirmacao antes de remover slide do banco
- * Controle de visibilidade: flag 'ativo' determina se slide aparece no slideshow publico
- * Ordenacao: campo 'ordem' controla a sequencia de exibicao

Acesso:

A rota /admin/slides e protegida. O AuthProvider verifica se o usuario possui isAdmin === true (baseado em app_metadata.role ou user_metadata.role === 'admin'). O link 'Gerenciar slides' so aparece no Header para administradores.

12. Estilizacao e Tema

A estilizacao utiliza **Tailwind CSS v4** com o plugin tw-animate-css. O tema e definido no arquivo styles.css com variaveis CSS customizadas.

Fontes:

Fonte	Uso	Estilo
Cormorant Garamond	Titulos e brand	Display, serif
Barlow	Corpo do texto	Sans-serif, clean

Paleta de Cores:

Cor	Uso	Valor
Primary (Dark Navy)	Fundo principal, textos de titulo	oklch(0.10 0.03 250)
Accent (Gold)	Destaques, bordas, acentos	oklch(0.78 0.14 75)
Deep Gold	Variacao do accent	oklch(0.65 0.16 75)
Success Green	Feedback positivo	oklch(0.72 0.19 155)

Componentes de UI:

- * shadcn/ui estilo New York (mais compacto e refinado)
- * Suporte a dark mode via variaveis CSS
- * Componentes construidos sobre Radix UI primitives
- * Animacoes via tw-animate-css

13. Configuracao de Build e Deploy

Vite + TanStack Start:

O projeto utiliza **Vite 8.1.5** como bundler com o preset Cloudflare Module para gerar saida compativel com Cloudflare Workers. O Nitro v3 beta e o motor de SSR.

Server Entry (server.ts):

- * Entry point para Cloudflare Workers
- * Envolva o server entry do TanStack Start
- * Trata erros do h3: detecta respostas JSON genericas 500 e substitui por paginas de erro renderizadas
- * Utiliza error-capture.ts para recuperar detalhes originais dos erros

Start Entry (start.ts):

- * Inicializacao do TanStack Start com dois middlewares de requisicao
- * errorMiddleware: captura erros do servidor e renderiza pagina de erro HTML para erros nao-HTTP
- * csrfMiddleware: protecao CSRF para server functions

Saida de Build:

- * Diretorio .output/ contem build de producao
- * Cliente: arquivos estaticos para upload ao Cloudflare Pages
- * Servidor: worker script para Cloudflare Workers
- * Configuracao em .output/server/wrangler.json

14. Seguranca e Protecoes

Mecanismo	Descricao
CSRF Protection	Middleware csrfMiddleware protege server functions contra Cross-Site Request Forgery
RLS (Row Level Security)	Políticas de banco de dados que garantem que usuarios so acessem seus proprios dados
Auth Session Validation	AuthProvider valida sessao existente via getUser() e escuta mudancas de estado
Admin Role Check	Verificacao em multiplos niveis: app_metadata.role e user_metadata.role === 'admin'
Owner Check	Apenas o criador de um produto pode edita-lo (a menos que seja admin)
Error Capture	Sistema de captura de erros que evita vazamento de stack traces ao cliente
Token Refresh	Supabase gerencia automaticamente o refresh de tokens de sessao
Invalid Session Handling	Sessao invalida resulta em signOut automatico e limpeza de estado

15. Resumo de Funcionalidades

Funcionalidade	Status	Detalhes
Catalogo de Produtos	Completo	Exibicao em grid com cards, filtros por categoria, busca por ID
Pagina de Detalhe	Completo	Imagem, precos, parcelamentos, avaliacoes, produtos relacionados
Carrinho de Compras	Completo	localStorage + sync cross-tab, selecao de itens, quantidade
Checkout via WhatsApp	Completo	Mensagem pre-preenchida, remocao automatica apos envio
Autenticacao	Completo	Login, cadastro, redefinicao de senha via Supabase
Controle de Acesso	Completo	Publico, autenticado, proprietario, administrador
Cadastrar Produto	Completo	Usuarios autenticados adicionam produtos ao catalogo
Editar/Excluir Produto	Completo	Proprietario ou admin podem editar/excluir
Gerenciar Slides	Completo	CRUD admin para slides do slideshow da home
Slideshow Animado	Completo	Carrossel automatico na home com 5s de intervalo
Design Responsivo	Completo	Suporte mobile via media queries + hook useIsMobile
Dark Mode	Completo	Variaveis CSS para tema escuro
SSR (Server-Side Rendering)	Completo	TanStack Start com Nitro + Cloudflare Workers
Error Handling	Completo	Paginas de erro, error boundary, captura de erros servidor
CSRF Protection	Completo	Middleware de protecao para server functions

Conclusao: O Lunar Shop e uma aplicacao e-commerce completa e funcional, construida com uma stack moderna e robusta. A arquitetura segue padroes saudaveis de desenvolvimento React com separacao clara de responsabilidades. O sistema de autenticacao e controle de acesso e bem estruturado, e o uso de Supabase simplifica significativamente o backend. O checkout via WhatsApp e uma solucao pratica para pequenos negocios brasileiros.